

HARVEST HAVOC SOURCE MANUAL

Absolute Engine Specification & Technical Documentation

Full Comprehensive Mapping of State, CSS Variables, Functions, Layout, and Scoring Algorithms

Document Version:	v1.0.0 (Complete System Breakdown)
Deployment Footprint:	Autonomous Client-Side Single Page Application (SPA)
Engine Stack:	Vanilla ECMAScript 6 / HTML5 Live Messaging / CSS Variables
Target Application:	2v2 Sports, Robotics, and Alliance-Based Competition Trackers

1. UI Layout Structure & CSS Design Token System

The system utilizes a custom-engineered UI framework constructed completely via vanilla HTML5 elements, inline CSS variables, and dynamic attributes. Themes are applied directly on the root component node using data attribute hooks, avoiding heavy runtime compilation stylesheets.

| 1.1 Core Styling & Palette Tokens (CSS Variables)

The interface relies on two distinct design tokens (Light and Dark mode) that transition instantly via CSS parameters when mutated on the parent DOM. The exact hexadecimal variables from the codebase are listed below:

Token Name	Standard Light Canvas Mode	Data-Dark Override Mode	Architectural Purpose
--bg	#ECEAE4	#111009	Primary system layout viewport background canvas
--card	FFFFFF	#1E1C18	Content wrappers, inputs, modals, and panel items
--border	#E0DDD6	#2D2B26	Structural separation borders and list item boundaries
--border2	#CBC8BC	#3D3A33	High-contrast element focus boundaries and form states
--text	#1A1917	#EDEBE5	Primary text canvas color for absolute maximum legibility
--text2	#6B6860	#A09C90	Secondary metadata descriptions, metrics, and captions
--text3	#A09C93	#6B6760	Disabled, placeholder, and empty-state text notes
--acc	#B91C1C	#F87171	Primary Accent color, brand markings, and highlights
--acc-h	#991B1B	#FCA5A5	Hover configurations for active interactive actions
--acc-bg	#FEF2F2	#3D0A0A	Background containers highlighting accent contexts
--acc-t	#7F1D1D	#FCA5A5	Text coloring used inside light accent backgrounds
--gold	#D97706	#FBBF24	Status warning displays, warnings, and special criteria
--green	#15803D	#4ADE80	Success tracking alerts, active clocks, additions
--blue	#1D4ED8	#60A5FA	Blue alliance mapping markers and tracking info
--red	#DC2626	#F87171	Red alliance mapping markers and tracking info

1.2 Dynamic Static Palette Lists

To differentiate participating groups dynamically, the engine defines a hardcoded array literal named `COLORS` consisting of exactly fourteen (14) high-contrast hexadecimal palette indices mapped sequentially during runtime insertion loops:

```
const
COLORS=['#B91C1C', '#1D4ED8', '#15803D', '#7C3AED', '#D97706', '#0E7490', '#BE185D', '#4338CA', '#047857',
```

2. Mathematical Formulation of Scoring Matrices

Harvest Havoc implements a complete three-tier game model separating Autonomous, Teleoperated, and End Game actions. Points computed across both the primary interface panel and secondary scoreboard mirrors evaluate according to a rigid algebraic schema.

2.1 Component Point Assignment Engine

The total score generated by an alliance (either **red** or **blue**) within a specific match is computed through the structural function **SC(r)**. The rigorous mathematical equation representing this valuation formula maps as follows:

Score Evaluation Function:

$$\text{Total Score} = (\text{auto_leave} \times 5) + (\text{auto_berry} \times 12) + (\text{auto_apple} \times 20) + (\text{tele_berry} \times 3) + (\text{tele_apple} \times 5) + (\text{cr_mixed} \times 12) + (\text{cr_berry} \times (18 + M_{\text{berry}})) + (\text{cr_apple} \times (24 + M_{\text{apple}})) + (\text{climb_lo} \times 30) + (\text{climb_hi} \times 50)$$

2.1.1 Structural Crate Bonus Multipliers (M_{berry} and M_{apple})

The score engine introduces programmatic logic altering crate valuations based on specific thresholds. These dynamic bonuses are derived as follows:

- **Berry Crate Multiplier Component (M_{berry}):** If the count of raw **tele_berry** items placed meets or exceeds exactly twelve (12), an additive bonus of **+6 points** is appended to the base configuration of *every* recorded berry crate.

If **tele_berry** ≥ 12 , then $M_{\text{berry}} = 6$; else $M_{\text{berry}} = 0$.

- **Apple Crate Multiplier Component (M_{apple}):** If the count of raw **tele_apple** items placed meets or exceeds exactly eight (8), an additive bonus of **+8 points** is appended to the base configuration of *every* recorded apple crate.

If **tele_apple** ≥ 8 , then $M_{\text{apple}} = 8$; else $M_{\text{apple}} = 0$.

2.2 Programmatic Evaluation Routine

The code processes this mathematical breakdown using a highly optimized, client-side JS evaluation statement:

```
function SC(r) {
  if(!r) return 0;
  let s = 0;
  s += (r.auto_leave || 0) * 5;
  s += (r.auto_berry || 0) * 12;
  s += (r.auto_apple || 0) * 20;
  s += (r.tele_berry || 0) * 3;
  s += (r.tele_apple || 0) * 5;
  s += (r.cr_mixed || 0) * 12;

  let bBonus = (r.tele_berry || 0) >= 12 ? 6 : 0;
  s += (r.cr_berry || 0) * (18 + bBonus);
```

```

let aBonus = (r.tele_apple || 0) >= 8 ? 8 : 0;
s += (r.cr_apple || 0) * (24 + aBonus);

s += (r.climb_lo || 0) * 30;
s += (r.climb_hi || 0) * 50;
return s;
}

```

3. Complete JavaScript Function & Interface API Mapping

This section documents every function block present inside the Harvest Havoc application script layout sequentially, outlining inputs, operations, and state dependencies.

3.1 Persistence, Theming, and Dialog Management

- **save()** : Synchronously updates browser cache storage by stringifying the state object **S** into key **'hhf'**.
Dependencies: **localStorage, S**
- **load()** : Reads cache records from keys **'hhf'**, **'hh_v3'**, or **'hh_v2'** sequentially to preserve backwards compatibility. If empty, instantiates empty object skeletons.
Dependencies: **localStorage, S**
- **applyDark(d)** : Modifies DOM presentation directly by adding attribute tag **data-dark="1"** or **"0"** to the core container element **#app**.
Arguments: **d (Boolean)**
- **toggleDark()** : Inverts theme flags, invokes **applyDark()**, and persists choice to cache storage.
Variables Tracked: **S.darkMode**
- **initDark()** : Evaluates default configurations on initial startup, querying system browser preference strings via **window.matchMedia('(prefers-color-scheme:dark)')** if custom configs do not exist.
- **OM(id) / CM(id)** : Open Module / Close Module handlers that add or strip the **'open'** visibility wrapper class from custom modal background structures.
Arguments: **id (String matching target DOM element identifier)**
- **doClear(w)** : Structural deletion algorithm tracking choice paths. If argument is **'scores'**, clears the results collection and resets status properties to pending. If **'sched'**, wipes match listings entirely. If **'all'**, completely clears team lists along with all historical match data.
Arguments: **w ('scores' | 'sched' | 'all')**

3.2 Navigation & Tab Routing Systems

- **P(id)** : Central SPA application controller. Removes the visibility modifier class **'active'** across all layout pages, hides inactive components, and applies active styling markers to the selected navbar layout buttons. Triggers downstream view updates for **'queue'**, **'score'**, **'rankings'**, **'results'**, and scoreboard elements.
Arguments: **id ('setup' | 'queue' | 'score' | 'rankings' | 'results' | 'sb')**
- **showTab(g, t, el)** : Embedded view toggle tracking sub-tab displays (such as switching sub-tables between Overall and Qual-Only rankings).
Arguments: **g (Group String Prefix), t (Target Mode ID), el (Element Pointer Handle)**

- `ss(e1, msg, type)` : Status notification renderer. Appends timed response dialog alerts (e.g. "Saved!") within specified container nodes, clearing notifications via `setTimeout` after exactly 2800 milliseconds.

3.3 Roster Management & Team Assembly

- `saveEvent()` : Captures the custom tournament name descriptor and standard target qualification parameters from input fields, updating metadata across all external display links.
- `checkGen()` : Validates core schedule limits. Disables generation triggers if total registered groups count falls below minimum threshold (*Teams* < 4).
- `gT(num)` : Registry locator routine. Iterates through team listings to fetch structural references matching key identifiers. Returns default mock containers if no matching team is found.
Arguments: `num` (String matching team index identifier)
- `addTeam()` : Core input routine. Evaluates field inputs, checks uniqueness constraints, dynamically pairs the team entry with a high-contrast entry color tracking tag from the palette array via modulus loops (`COLORS[S.teams.length % COLORS.length]`), and refreshes component displays.
- `removeTeam(num)` : Filters the active array data list to remove team elements matching the target identifier parameter.
- `startET(num)` / `cancelET(num)` / `saveET(num)` : Inline team editor controllers. Swaps list view lines with live input field items, tracking value corrections and propagating ID name updates through all historical listings, live queues, and deck allocations.

3.4 Automated Tournament Pairing Logic

The `genSched()` algorithm compiles round-robin-style qualification pairings without backend calculation resources. It utilizes a randomizing shuffle strategy designed to meet three target parameters: each team plays exactly *N* times, teammates and opponents vary constantly, and red/blue alliance assignments remain evenly balanced.

```
function genSched() {
  const n = S.teams.length;
  if(n < 4) return;
  const per = S.event.twPer || 5;
  const tArr = S.teams.map(t => t.num);

  // Instantiates tracking arrays counting absolute match occurrences per asset
  let matchPool = [];
  let totalMatchesNeeded = Math.ceil((n * per) / 4);

  // Creates an extended randomized pool layout
  let flat = [];
  for(let i=0; i<per; i++) {
    let copy = [...tArr];
    // Performs continuous inline Fisher-Yates array shuffling loops
    for(let j=copy.length-1; j>0; j--) {
      const k = Math.floor(Math.random() * (j + 1));
      [copy[j], copy[k]] = [copy[k], copy[j]];
    }
    flat.push(...copy);
  }

  // Segment structural elements from flat array into distinct 4-team slots
```

```

S.matches = [];
let matchId = 1;
for(let i=0; i<flat.length; i+=4) {
  if(i+3 < flat.length) {
    S.matches.push({
      id: matchId,
      num: matchId,
      type: 'qual',
      red: [flat[i], flat[i+1]],
      blue: [flat[i+2], flat[i+3]],
      status: 'pending'
    });
    matchId++;
  }
}

// Post-processing schedule verification loops
const pend = S.matches.filter(x => x.status === 'pending');
S.currentMatch = pend[0] || null;
S.onDeckMatch = pend[1] || null;

save();
initUI();
}

```

3.5 Complete Core Function Inventory

The remaining internal function hooks governing the scoring lifecycle are detailed in the following dictionary table:

Function Symbol	Argument Parameters	Algorithmic Operation & Internal Side Effects
<code>importCSV()</code>	None	Parses raw comma-delimited strings from user inputs, splits rows on newline boundaries, maps individual text columns into matching alliance objects, and inserts new match items into the active schedule queue.
<code>loadSamples()</code>	None	Clears existing data entries and populates the database structure with exactly twelve (12) standard sample team arrays to facilitate immediate pipeline testing.
<code>renderTeams()</code>	None	Generates and outputs raw HTML elements for the team roster panel, attaching dynamic control binds and injecting inline styles to display assigned colors.
<code>renderQueue()</code>	None	Renders listings across the dashboard view, dynamically coloring cards based on queue position (e.g. green for on-field, red for upcoming on-deck).
<code>callMatch(id)</code>	<code>id</code> (Integer)	Locates the target match object from the schedule listing, sets it as the active field item, configures the next item in line as on-deck, and broadcasts the updated state layout to the scoreboard viewport.
<code>skipMatch(id)</code>	<code>id</code> (Integer)	Removes the target item from its current position in the schedule array and appends it to the very end of the list, preserving its data properties while shifting structural queue indices.
<code>openAddMatch()</code>	None	Populates dropdown selection elements inside custom popup modules with lists of all valid teams, preparing the interface to add manual override match records.
<code>confirmAM()</code>	None	Validates dropdown inputs to ensure all selected teams are distinct. Appends a new item flagged as an <code>'extra'</code> match type to the schedule queue and hides the modal dialog box.
<code>completeMQ()</code>	None	Loops through all outstanding items remaining in the tournament queue, automatically setting their status flags to complete to clean up remaining scheduling lists.
<code>adj(ctx, side, k, v, max)</code>	<code>ctx, side, k, v, max</code>	Processes increment and decrement button updates for alliance scores. Clamps values to prevent negative inputs (<code>Count < 0</code>) and enforces specified upper limits (such as a maximum value of 2 for taxi bonuses). Updates the score tracking state and recalculates totals.
<code>recalc(ctx)</code>	<code>ctx</code> (Context String)	Queries value markers across scoring input panels, calculates totals using the <code>SC(r)</code> scoring function, updates the main interface display elements, and conditionally adjusts crate score values if bonus thresholds are met.
<code>saveScores(ctx, mid)</code>	<code>ctx, mid</code>	Extracts input parameters from the interface view, aggregates scores for both alliances, bundles the data into a standardized results record with a precise millisecond timestamp, and inserts the record into the results list.
<code>doSubmit(fromEdit)</code>		

Function Symbol	Argument Parameters	Algorithmic Operation & Internal Side Effects
	<code>fromEdit</code> (Boolean)	Validates score entries for active matches. Updates the status flag of the submitted match item, shifts subsequent queue listings forward, and refreshes the leaderboard and analytics views.
<code>doSubmitFinish()</code>	None	Saves active score records, pauses the countdown timer, locks live score displays, automatically triggers the celebration splash view on the scoreboard for 5 seconds, and then routes the audience display to the final results view.

4. Structural Display Pipeline & Inter-Window Payload

The communication channel between the main administration panel and the secondary audience display functions via memory object sharing, avoiding the overhead of local storage polling loops or network sockets.

4.1 Data Structure Specification (The Payload Object)

When updates occur on the administration console, a synchronized snapshot object is generated and passed directly into the child display container via the `receivePayload(p)` routine. The payload snapshot contains the following data fields:

```
{
  state: "pre" | "live" | "splash" | "post", // Current scoreboard display view state
  event: { name: "Event Title", twPer: 5 }, // Global tournament metadata properties
  currentMatch: { id: 1, num: 1, red: [...], blue: [...] }, // Data context for the active match
  onDeckMatch: { id: 2, num: 2, red: [...], blue: [...] }, // Data context for the upcoming match
  match
    timerText: "2:40", // Pre-formatted time duration string
    timerMax: 180, // Target duration value for the selected mode
    timerVal: 160, // Active duration value remaining in seconds
    timerRunning: true, // State flag indicating if the timer loop is
  active
    timerEnabled: true, // Visibility configuration flag for the timer view
    livePaused: false, // Lock setting for real-time live score updates
    scores: { redScore: 45, blueScore: 60 }, // Computed point values for both alliances
    rankings: [ ... ] // Sorted list array tracking current team
  standings
}
```

4.2 Scoreboard Rendering & View State Logic

The audience display window processes incoming payload snapshots through the `buildSBHTML(p)` routine. This function dynamically generates complete HTML layouts based on the active display state, applying the following layout logic:

- **Pre-Match Display Layout (`p.state === 'pre'`):** Renders large alliance panels displaying team names and assignment colors alongside a list detailing the next upcoming match to organize field queues.

- **Live Match Interface (`p.state === 'live'`):** Renders real-time scoring bars, team identification markers, and a large central countdown timer colored to reflect match urgency (e.g. transitions to orange when ***Time ≤ 30s***, and red when ***Time ≤ 10s***).
- **Splash Overlay Celebration View (`p.state === 'splash'`):** Renders a high-impact celebration layout featuring flashing colors, custom win banners, and bold labels highlighting the winning alliance along with final point scores.
- **Post-Match Leaderboard Display (`p.state === 'post'`):** Renders a comprehensive tournament leaderboard ranking teams based on performance metrics, allowing the audience to track current event standings.